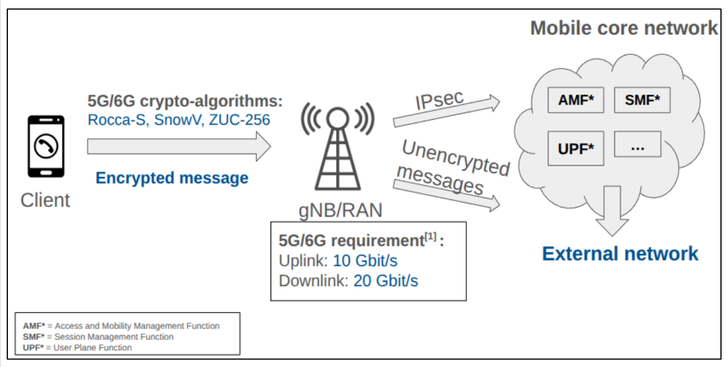
Designing size-aware cryptographic primitives for FPGA-based accelerators

Aditya Peer, Neeraj Kumar Yadav, Rohan Sudhir Basugade, Agamdeep Singh, Siddhant, Sumit Darak and Rinku Shah
IIIT Delhi

IndoSys 2026

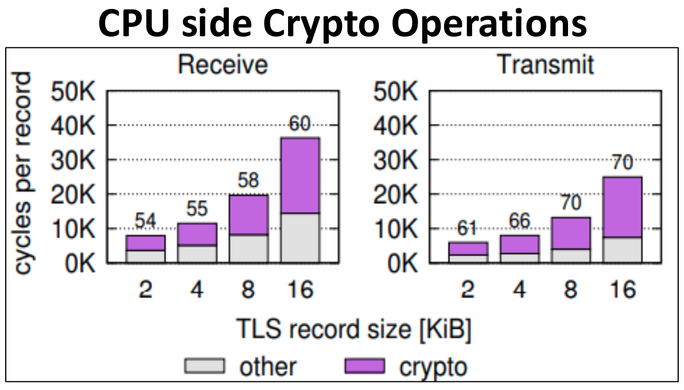
Background

Cryptographic primitives use at Radio Access Network



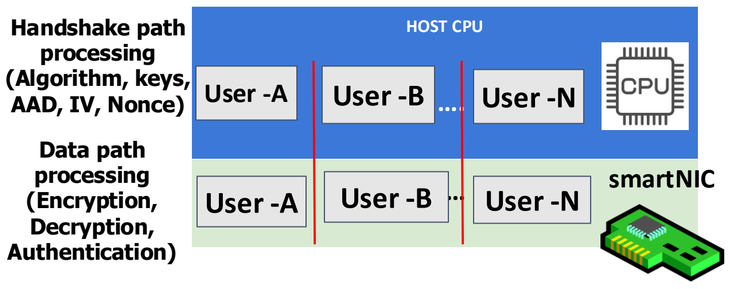
Other Applications:
- Telecom Networks , Military Applications, Data Centers

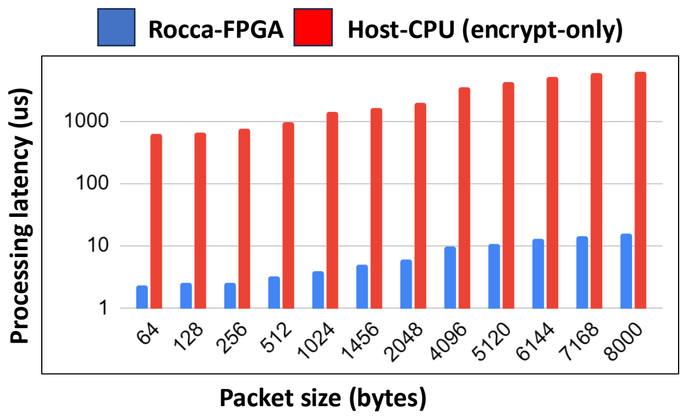
CPU side Crypto Operations



Encryption/decryption[1]
- Simple TCP bandwidth test
- Modified Iperf to use OppenSSL
- 60-74% CPU cycles spent for TLS TX/RX

Server accelerates Data Centers applications by leveraging Smart NICs





Rocca-S processing latency
❖Offload < Host processing
➢Up to 99.75%

State-of-the-art solutions

Fixed function ASIC solutions Not flexible	Reconfigurable solutions Flexible
• Nvidia's Bluefield[2] • AMD Pensando's DSC • Netronome CX NICs	• FPGA-based solutions • Chacha on programmable NIC[3] • AES on programmable Tofino switch

Limitations of the State-of-the-art

- CPUs are general-purpose, may lead to performance bottlenecks
  - CPU Specialized Accelerators (AES-NI)
    - Consumes lot of CPU cycles !
- ASICs are fast but fixed function
- Programmable smart NICs
  - Reconfigurable but have limited resources
  - Focus on individual algorithm

The above solutions are optimized to be resource , power, throughput efficient but are not workload aware !

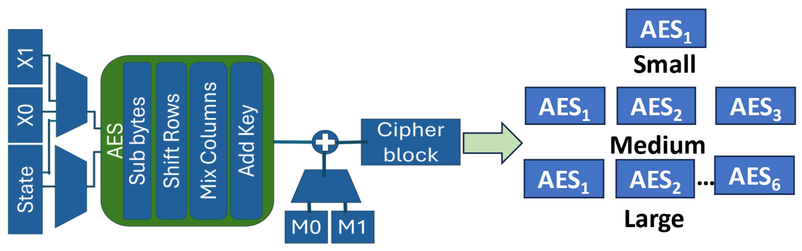
Resource, power or throughput trade-off for FPGAs ?

- As 5G telecom algorithms continue to evolve, there is a need for:
  - Programmability
  - Configurability

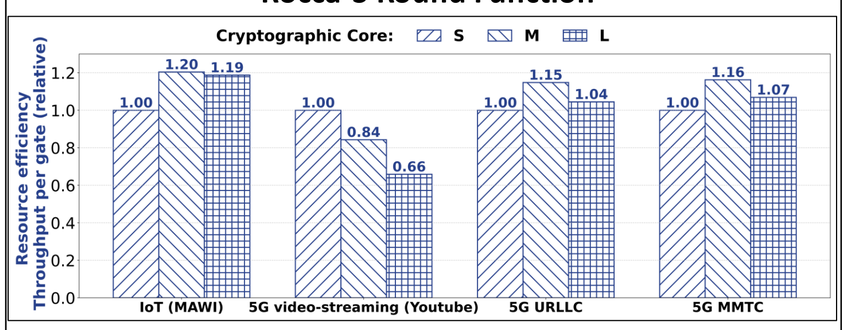
Earlier FPGA solutions focused on either:
- High throughput => Using more resources
- Resource efficient(Reusing the components) =>Increase in latency (sequential processing)

Our Problem Statement & Design

Cryptographic Core Design = {Small, Medium$_n$, Large}



Rocca-S Round Function



Rocca-S's resource efficiency across cryptographic core variants for 5G applications relative to the cryptographic core, "S".

- ➢M Cryptographic core => Resource efficient ↑ for 5G datasets
- ➢S Cryptographic core => Resource efficient ↑ for 5G video streaming dataset
- ➢L Cryptographic core ≅ M cryptographic core for IOT dataset

Design cryptographic primitive offload framework that to :
- Overcome trade-off between performance & resource utilization
- Reconfigurable
  - Dynamically reconfigurable (DPR)
- Workload-aware
- Asymmetric Replication → Workload-aware

Scheduler Design

Scheduler Toy example

Arrival time	0	0	0	2	0	1	2	2	3
Request ID (rij)	r_{11}	r_{12}	r_{13}	r_{14}	r_{21}	r_{22}	r_{23}	r_{24}	r_{25}
Service time	1	1	2	2	1	1	1	2	1
Request len (ri.len)	300	500	2000	8000	100	400	200	4000	500
Rank (ri.rank)	4	3	2	0	4	4	4	1	3

Requests arrived at the NIC; rij= jth request of flow i

FlexMesh$_{nWC}$

Virtual time		1	2	3	4	5	6	7
Cryptographic core (C_1)		r_{11}	r_{21}	r_{12}	r_{14}		r_{23}	r_{25}
Cryptographic core (C_2)		r_{13}		r_{22}	r_{24}			
Wire Output		r_{11}	r_{21}	r_{13}	r_{12}	r_{22}	r_{14}	r_{24}	r_{23}	r_{25}

Non-work conserving (2 idle) , OOO Size=14.5 KB, total time = 7

FlexMesh

Virtual time		1	2	3	4	5	6
Cryptographic core (C_1)	HEAD	r_{11}	r_{21}	r_{12}	r_{14}		r_{23}
Cryptographic core (C_2)		r_{13}		r_{22}	r_{24}		r_{25}
Wire Output	HEAD	r_{11}	r_{21}	r_{13}	r_{12}	r_{22}	r_{14}	r_{24}	r_{23}	r_{25}

Work-conserving, OOO Size = 14.5 KB, total time = 6

FlexMesh$_p$

Virtual time		1	2	3	4	5	6
Eligible requests (E1)	HEAD	---	r_{21}, r_{12}	r_{21}, r_{14}, r_{23}		r_{21}, r_{23}, r_{25}	r_{21}, r_{23}
Rank ($rank_1$)		---	4, 3	4, 0, 4		4, 4, 3	4, 4
Cryptographic core (C_1)		r_{11}	r_{12}	r_{14}		r_{25}	r_{21}
Eligible requests (E_2)		---		r_{22}, r_{24}		r_{22}	
Rank ($rank_2$)				4, 1		4	
Cryptographic core (C_2)		r_{13}		r_{24}		r_{22}	r_{23}
Wire output	HEAD	r_{11}	r_{12}	r_{13}	r_{14}	r_{24}	r_{22}	r_{25}	r_{21}	r_{23}

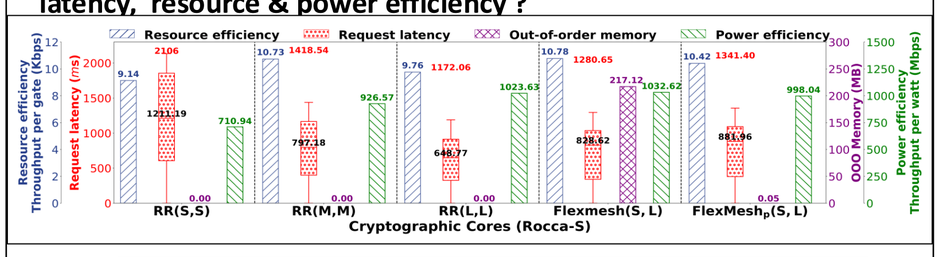
Work-conserving, OOO size = 4.9 KB, total time = 6

rij – jth request of flow i, Out-of-order, Work-conserving, Idle

Experimental Setup & Results

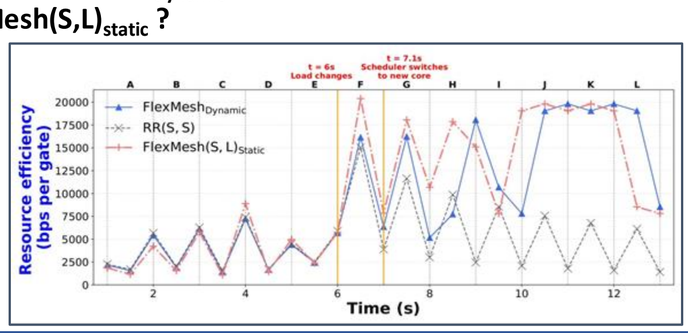
- ➢Zynq UltraScale+ MPSoC ZCU106)
  - Rocca-S implementation on (ZCU-106+ board)

➢ **How different scheduling policies impact OOO memory, request latency, resource & power efficiency ?**



FlexMesh => Resource ↑, Power ↑ & Lower request Latency ↑

➢ **How do FlexMesh$_{dynamic}$ resource efficient w.r.t to RR(S,S) & FlexMesh(S,L)$_{static}$?**



- ➢FlexMesh$_{dynamic}$ ≃ RR(S,S) > FlexMesh(S,L)$_{static}$ (t < 6 seconds)
- ➢FlexMesh$_{dynamic}$ ≃ FlexMesh(S,L) > RR(S,S) (t > 7.1 seconds)

Conclusion

➢We present FlexMesh, an in-network cryptographic primitive that leverages the workload request size characteristics to design cryptographic accelerators with high resource and power efficiency, without compromising the performance.

➢We design and implement variable-sized cryptographic primitives for next-generation mobile networks (i.e., Rocca-S), and a programmable scheduler that prioritizes flow order and work-conservation.

References

[1] Boris Pismenny, Haggai Eran, Aviad Yehezkel, Liran Liss, Adam Morrison, and Dan Tsafrir. 2021. Autonomous NIC Offloads
[2] 2021. NVIDIA BLUEFIELD-3 DPU PROGRAMMABLE DATA CENTER INFRASTRUCTURE ON-A-CHIP. https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/documents/datasheet-nvidia-bluefield-3-dpu.pdf.
[3]Shaguftha Zuveria Kottur, Krishna Kadiyala, Praveen Tammana, and Rinku Shah. 2022. Implementing ChaCha Based Crypto Primitives on Programmable SmartNICs.

Have a Chai with the authors

Aditya Peer
PhD Scholar
Dr. Rinku Shah
Assistant Professor

Paper link

GitHub code available